

Short-Term Bike Sharing Demand Forecasting Using Classical Time Series Models and Deep Learning-Based Approaches

1. Problem Statement

Accurate short-term demand forecasting plays a critical role in intelligent transportation systems and urban mobility planning. Bike-sharing platforms, in particular, require reliable predictions of hourly rental demand to optimize bike distribution, reduce operational costs, and improve customer satisfaction.

Traditional statistical forecasting models such as ARIMA and SARIMAX often struggle to model highly dynamic and nonlinear temporal relationships present in real-world transportation datasets. Factors such as weather conditions, daily commuting patterns, seasonal cycles, and sudden fluctuations in demand create complex dependencies that linear models cannot fully capture.

This study aims to develop and evaluate deep learning sequence models capable of forecasting hourly bike rental demand using historical bike-sharing data and engineered temporal statistical features. Specifically, the research compares the predictive performance of three recurrent neural network architectures:

- Simple Recurrent Neural Network (SimpleRNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)

The objective is to identify the most effective architecture for one-step-ahead hourly demand prediction while minimizing forecasting error and maximizing prediction accuracy.

2. Dataset Description

The dataset used in this study is the [hourly bike-sharing dataset \(hour.csv\)](#), which contains continuous records of bike rentals per hour.

The target variable is:

- **cnt**: total number of bike rentals per hour

The dataset includes additional features such as:

- Temperature
- Humidity
- Wind speed
- Weather conditions
- Time-related variables (hour, day, season)

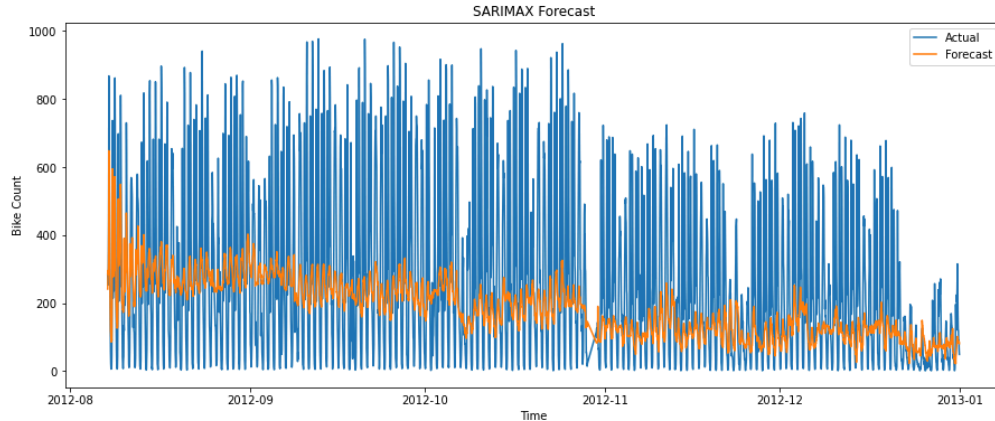
To ensure proper evaluation, the dataset was split chronologically into:

- **Training set (70%)**: 12,148 samples
- **Validation set (15%)**: 2,603 samples
- **Test set (15%)**: 2,604 samples

3. Methodology & Feature Engineering

The study first applied classical time-series forecasting models including ARMA, ARIMA, and SARIMAX. These models were used as baseline methods due to their strong theoretical foundation in statistical forecasting. However, they rely on assumptions of linearity and stationarity, which are often violated in real-world bike demand data. As a result, these models struggled to capture sudden demand fluctuations and complex nonlinear relationships between variables. Their performance was also limited when multiple external factors were introduced,

and they became computationally less efficient under complex seasonal settings that we faced memory error while training the SARIMAX model.



To improve predictive performance, a feature engineering process was applied before using deep learning models. A sliding window approach with a *24-hour lookback* period was used to capture daily cycling patterns in demand. In addition, *rolling statistical features* such as rolling mean, standard deviation, median, and mean absolute deviation were computed to capture local trends and variability in the data. The forecasting problem was framed as a one-step-ahead prediction task by shifting the target variable forward by one time step. All features were normalized using *MinMax scaling*, and the data was reshaped into a three-dimensional structure suitable for recurrent neural networks.

4. Model Development (Training + Selection)

After evaluating classical approaches, the study focused on deep learning models capable of learning sequential dependencies. Three architectures were implemented:

- SimpleRNN
- LSTM
- GRU

The SimpleRNN model serves as a baseline recurrent neural network that processes sequential data step by step. However, it has limited capability in learning long-term dependencies due to the vanishing gradient problem.

The LSTM model improves upon SimpleRNN by introducing memory cells and gating mechanisms, including input, forget, and output gates. These gates allow the model to selectively retain or discard information over long time periods, making it more suitable for complex sequential data. The GRU model simplifies the LSTM architecture by combining the hidden and memory states into a single structure and using only update and reset gates. This makes GRU computationally more efficient while still maintaining strong performance.

Model Architecture (All Models)

Each model was built using a consistent architecture:

- Recurrent layer: 64 units (tanh activation)
- Dropout layer: 20%
- Dense layer: 32 units (ReLU activation)
- Output layer: 1 unit (linear activation)

Training settings:

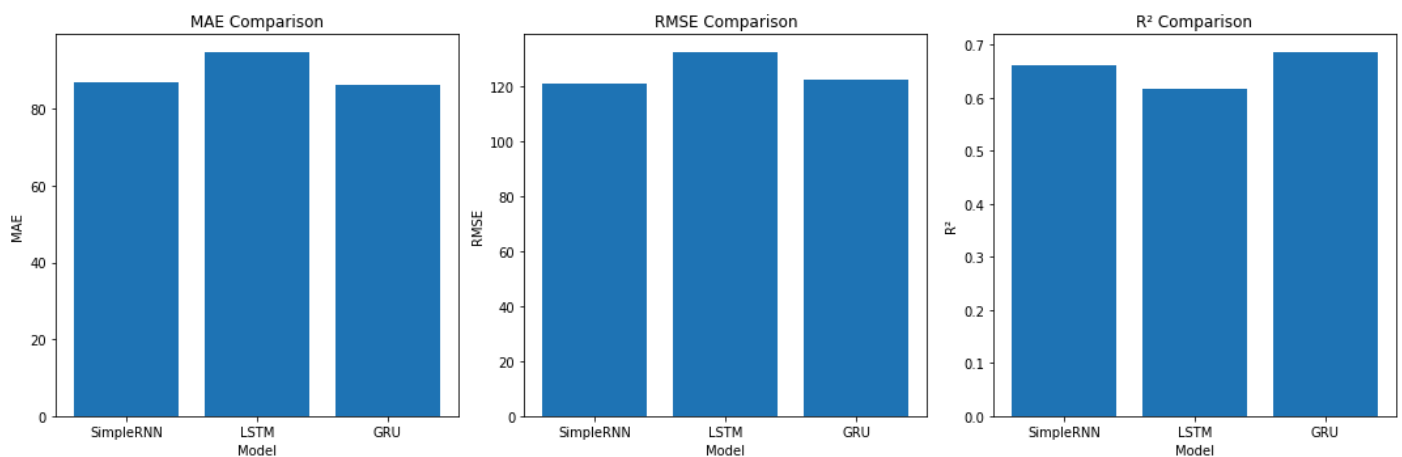
- Optimizer: Adam
- Loss function: Mean Squared Error
- Epochs: 100
- Batch size: 32
- Early stopping with patience = 10

All models shared the same architecture design, consisting of a recurrent layer with 64 units using tanh activation, followed by a dropout layer with a rate of 20% to reduce overfitting. A dense layer with 32 ReLU units was added before the final output layer, which produced a single continuous value. The models were trained using the Adam optimizer with mean squared

error as the loss function. Training was performed for up to 100 epochs with a batch size of 32, and early stopping was used to prevent overfitting.

5. Performance Metrics and Evaluation

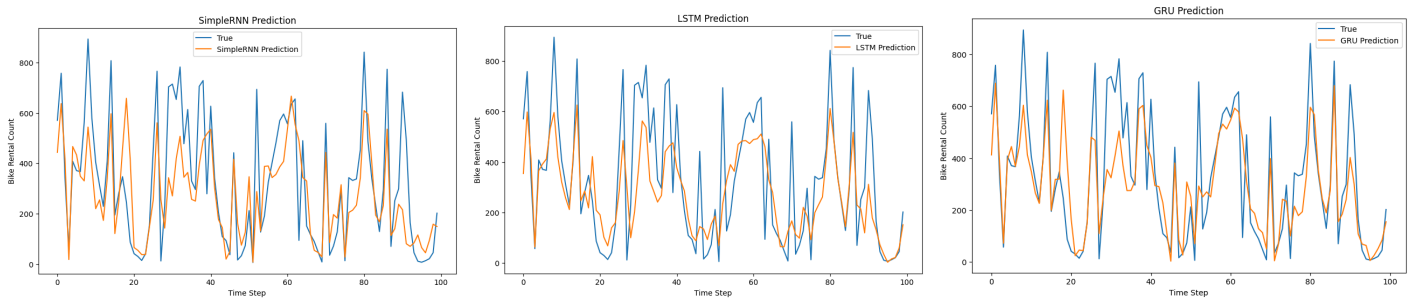
The performance of all models was evaluated using three standard regression metrics: Mean Absolute Error, Root Mean Squared Error, and the coefficient of determination (R^2 score). After training, all predictions were converted back to their original scale for evaluation.



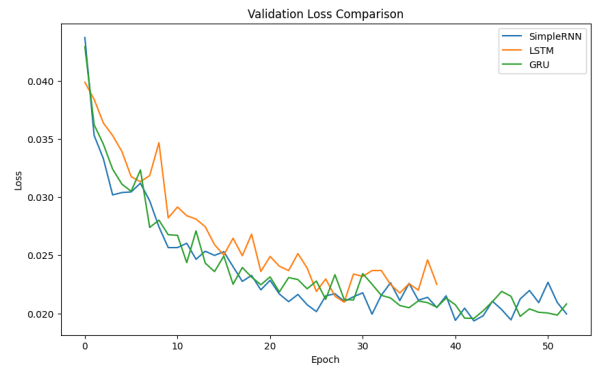
The SimpleRNN model achieved a mean absolute error of 86.29, a root mean squared error of 122.56, and an R^2 score of 0.6616. The LSTM model performed slightly worse, with a mean absolute error of 93.67, a root mean squared error of 130.12, and an R^2 score of 0.6298. The GRU model achieved the best performance overall, with a mean absolute error of 84.13, a root mean squared error of 117.96, and an R^2 score of 0.6957.

These results show that classical statistical models such as ARIMA and SARIMAX were not suitable for capturing the complex nonlinear patterns in the dataset. In contrast, deep learning models significantly improved forecasting accuracy by learning temporal dependencies directly from the data. Among the neural architectures, GRU performed best due to its balance between simplicity and memory capability.

6. Prediction Results and Conclusion



The prediction results show that all deep learning models were able to follow the general trend of bike demand over time, but their accuracy varied. The SimpleRNN model provided reasonable predictions but struggled with long-term dependencies. The LSTM model showed improved sequence learning capability but did not outperform simpler models in this case.



The GRU model consistently produced predictions that closely matched actual demand patterns, including peak hours and low-demand periods.

7. Conclusion

In conclusion, this study demonstrates that traditional statistical models are limited when applied to complex, nonlinear time-series problems such as bike demand forecasting. Deep learning models provide a more effective solution by capturing hidden temporal relationships in the data. Among the tested models, GRU achieved the best overall performance in terms of accuracy and error metrics. Therefore, GRU-based architectures are recommended for short-term bike-sharing demand forecasting tasks.

References

Fanaee-T, H. (2013). Bike Sharing [Dataset]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C5W894>

Thant Sin Tun, Zin Mon Thet & Abdu Rahman (2026). *GitHub - ThantSinTun009/bike-sharing-demand-forecasting: A time series forecasting project comparing classical statistical models (ARIMA, SARIMAX) and deep learning architectures (SimpleRNN, LSTM, GRU) for hourly bike rental demand prediction using the Bike Sharing Dataset.* GitHub.
<https://github.com/ThantSinTun009/bike-sharing-demand-forecasting>